VRS Express to VRS AI Integration Handoff

AHS authorization API, webhook, security, and staging acceptance contract

Item	Value
Audience	VRS Express and VRS AI implementation teams
Status	Implementation handoff - staging activation pending
Prepared	2026-07-29
Contract basis	API Contract V5 and the approved VRS Express / VRS AI ownership model
VRS Express status	Source merged and published; bridge activation pending configuration and acceptance
Machine-readable contract	vrs-express-vrs-ai-bridge-openapi.yaml

This document defines the implementation interface between VRS Express and VRS AI. VRS Express source is merged and published. The bridge remains disabled for live traffic until staging configuration, the additive development schema, and joint acceptance checks are complete. No live keys or secrets are included in this package.

1. What VRS AI needs to implement

VRS AI must:

1. Host `POST /vrs/authorize` at the confirmed VRS AI staging and production origins.
2. Authenticate Express with the static `X-API-Key` header.
3. Accept the strict V5 authorization payload and return an asynchronous call identifier.
4. Own call admission, Twilio/AHS execution, prompt construction, recordings, and AI call state.
5. Send signed V5 result events to the `callback_url` and with the `callback_secret` supplied in each request.
6. Retry transient callback failures without changing the event ID or body.

VRS Express already implements the human approval gate, outbound request, callback verification, ticket/SMS application, event idempotency, and final callback forwarding to the upstream workflow.

2. Confirmed runtime flow

```
VRS Express
  -> receives and validates the authorization request
  -> normal pending Express ticket
  -> VRS agent reviews the complete business context
  -> agent selects "Approve & Send to VRS AI"
  -> Express POSTs /vrs/authorize to VRS AI
  -> VRS AI calls AHS
  -> VRS AI POSTs signed result to Express callback_url
  -> Express applies the normal ticket outcome and technician SMS
  -> Express signs and forwards the result to the upstream workflow
```

The `POST /vrs/authorize` request received by VRS AI occurs only after human approval in VRS Express. VRS AI should still treat its `202` response as asynchronous admission, not proof that dialing has started or completed.

3. System ownership

System	Owns
VRS Express	Authorization intake, human review, outbound AI dispatch, result verification, ticket/SMS processing, and upstream forwarding
VRS AI	Admission limits, prompt construction, AHS/Twilio call, recordings, call lifecycle, signed callback production and retries

Phase 1 is American Home Shield authorization only. First American is a separate portal workflow and is not part of this interface. Choice, OEM, and Assurance are also outside this increment.

4. VRS AI environments

Environment	VRS AI origin	Express calls
Staging / development	https://vrs-dev.stage.nextgen.shs.com	POST https://vrs-dev.stage.nextgen.shs.com/vrs/authorize
Production	https://vrs.prod.nextgen.shs.com	POST https://vrs.prod.nextgen.shs.com/vrs/authorize

Express uses a 15-second HTTP timeout. VRS AI must persist/admit the work and return the asynchronous response within that window.

5. Express to VRS AI authorize request

5.1 HTTP request

```
POST /vrs/authorize HTTP/1.1
Host: vrs-dev.stage.nextgen.shs.com
Content-Type: application/json
X-API-Key: <VRS-AI-API-KEY>
```

The API key must be exchanged through an approved secret channel. It must never appear in source control, screenshots, tickets, or application logs.

5.2 Example body

```
{
  "service_order": "13538277",
  "tech_id": "0123456",
  "tech_phone": "+12145550199",
  "tech_name": "Example Technician",
  "reason_for_call": "authorization",
  "callback_url": "https://<express-stage-host>/api/integrations/vrs-ai/v1/webhook",
  "callback_secret": "<EXPRESS-SUPPLIED-HMAC-SECRET>",
  "parts": [
    {
      "part_id": "WPW10503278",
      "part_description": "Electronic Control Board",
      "sell_price": 245.0,
      "part_function": "Controls wash cycle timing and motor speed."
    }
  ],
  "customer_name": "Example Customer",
  "customer_address": "742 Evergreen Terrace, Hagerstown, MD 21740",
  "service_address": "742 Evergreen Terrace, Hagerstown, MD 21740",
  "brand": "Whirlpool",
  "model_number": "WTW5000DW1",
  "serial_number": "C31290822",
  "appliance_type": "Washer",
  "appliance_age": 4,
  "labor_amount": 120.0,
```

```
  "tax_amount": 15.6,
  "part_price": 245.0,
  "appliance_location": "laundry room",
  "failure_cause": "normal-wear",
  "tech_recommendation": "repair",
  "repair_replace_rationale": "Board failure is isolated and repair is within the coverage threshold.",
  "appliance_grade": "mid",
  "partner_id": "ahs",
  "appliance_photos": [
    "https://srdm.example.com/photo_1.jpg"
  ],
  "comments": [
    {
      "tech_id": "0123456",
      "work_item": "failure_diagnosis",
      "text": "Diagnostic code F7E1.",
      "timestamp": "2026-05-12T13:55:00Z"
    }
  ]
}
```


Express passes the approved V5 business payload through unchanged, except that it replaces the upstream callback_url and callback_secret with the Express callback endpoint and Express callback secret.

5.3 Required fields and constraints

Unknown fields are not accepted at the Express boundary. VRS AI should implement the V5 names exactly; V4 aliases such as `end_level` and `diagnostic_photos` must not be used.

Field	Type and constraint
service_order	Required string, exactly 8 digits
tech_id	Required non-empty string, max 50
tech_phone	Required E.164 string
tech_name	Required non-empty string, max 255
reason_for_call	Required literal authorization
callback_url	Required clean HTTPS URL ending exactly /api/integrations/vrs-ai/v1/webhook
callback_secret	Required string, 16-512 characters; HMAC key for this callback
parts	Required array, at least one part
customer_name	Required non-empty string, max 255
customer_address	Required non-empty string, max 1000
service_address	Required non-empty string, max 1000
brand	Required non-empty string, max 100
model_number	Required non-empty string, max 100
serial_number	Required non-empty string, max 100
appliance_type	Required non-empty string, max 100
appliance_age	Required number, 0-100
labor_amount	Required USD number, 0-1,000,000, max 2 decimals
tax_amount	Required USD number, 0-1,000,000, max 2 decimals
part_price	Required USD number, 0-1,000,000, max 2 decimals
appliance_location	Required non-empty string, max 100

Field	Type and constraint
failure_cause	normal-wear, power-surge, impact, installation-error, abuse, age, or other
tech_recommendation	repair or replace
repair_replace_rationale	Required non-empty free text
appliance_grade	Optional: entry, mid, or top
partner_id	Optional; Phase 1 accepts ahs or american_home_shield. Missing means AHS.
appliance_photos	Optional array of up to 10 absolute URLs
comments	Optional array of up to 20 structured comments

Each `parts[]` entry requires:

Field	Type and constraint
part_id	Non-empty string, max 100
part_description	Non-empty string, max 500
sell_price	USD number, 0-1,000,000, max 2 decimals
part_function	Non-empty string, max 200

Each `comments[]` entry requires:

Field	Type and constraint
tech_id	Non-empty string, max 50
work_item	failure_diagnosis, tech_recommendation, appliance_features, customer_history, tech_observation, or other
text	Non-empty string, max 2000
timestamp	ISO-8601 date-time with an offset

5.4 V5 data handling requirements

- Use `parts[].sell_price` as the primary catalog price.
- Keep top-level `part_price` separate. It is the technician's total selected or override part amount, not a replacement for each part's catalog price.
- Customer and appliance fields are already supplied. Do not require a Snowflake fetch for those fields.
- VRS AI still owns the `cta_no` lookup from `IH_SERVICEORDER_FULLDETAIL_TBL`, keyed by `service_order`. Express does not send `cta_no`.
- Use `appliance_type` as additive prompt context even when `model_number` is present.
- VRS AI must confirm the prompt placement of `repair_replace_rationale`; see Open Decision 1 below.

6. VRS AI authorize responses

6.1 Accepted

```
HTTP/1.1 202 Accepted
Content-Type: application/json

{
  "call_id": "vrs_ai_call_456",
  "service_order": "13538277",
  "state": "queued",
```

```
  "created_at": "2026-07-24T17:20:00Z"
}
```

`call_id` is VRS AI's provider call identifier. It must be globally unique and stable. Express stores it separately from the public Express bridge ID. Every callback from VRS AI must use this value in `callId`.

6.2 Duplicate active call

```
HTTP/1.1 409 Conflict
Content-Type: application/json

{
  "error": "DUPLICATE_CALL",
  "message": "A call is already in progress for service order 13538277",
  "existing_call_id": "vrs_ai_call_456",
  "state": "in_progress"
}
```

Express treats `409` as a successful reattachment only when `existing_call_id` is present.

6.3 Rate limited

```
HTTP/1.1 429 Too Many Requests
Content-Type: application/json

{
  "error": "RATE_LIMITED",
  "message": "Tech 0123456 has exceeded 10 calls/hour",
  "retry_after_seconds": 1800
}
```

The V5 admission limits owned by VRS AI are:

- 10 requests per `tech_id` per hour
- 1 active call per `service_order`
- 100 globally concurrent active calls

6.4 Other response behavior

Status	Express behavior
202 with call_id	Accepted and Stage 1 approval completes
409 with existing_call_id	Accepted as duplicate/reattached
429	Dispatch remains failed/retryable for the agent
400	Dispatch fails; return actionable validation message
401 / 403	Dispatch fails; check API-key configuration
500 / 502 / 503 or timeout	Dispatch fails; do not claim call admission

For any non-contract response, include a safe human-readable `message`.

7. VRS AI to Express callback

VRS AI must POST to the exact callback_url received in the authorize request. Do not hardcode an upstream callback URL. Express owns the callback hop from VRS AI.

```
POST /api/integrations/vrs-ai/v1/webhook HTTP/1.1
Host: <express-host>
Content-Type: application/json
X-VRS-Signature: sha256=<64-lowercase-hex>
X-VRS-Event-Id: <stable-event-id>
X-VRS-Timestamp: <current-unix-epoch-milliseconds>
```

7.1 Common event envelope

Every event requires:

Field	Requirement
version	Integer literal 1
eventId	Stable event identifier, max 100; use UUID v4 in production
callId	Exact VRS AI call_id returned from /vrs/authorize
soNo	Exact 8-digit service_order from the request
occurredAt	ISO-8601 date-time with offset
state	One of the five Phase 1 event states below

Do not send `queued`, `dialing`, `connected`, or `awaiting_decision` callbacks in Phase 1. Express intentionally rejects those Phase 2 event shapes.

7.2 Approved

```
{
  "version": 1,
  "eventId": "8f3a5e10-58a4-4fe0-a3a1-95038a7e4ee6",
  "callId": "vrs_ai_call_456",
  "soNo": "13538277",
  "occurredAt": "2026-07-24T17:24:00Z",
  "state": "approved",
  "authNumber": "AHS-001",
  "authorizedAmount": 365.6,
  "durationSec": 187,
  "recordingUrl": "https://vrs-dev.stage.nextgen.shs.com/api/vrs/call/456/recording"
}
```

Required outcome fields: `authNumber`, `authorizedAmount`. `durationSec` and `recordingUrl` are optional.

7.3 Partial approval

```
{
  "version": 1,
  "eventId": "77762099-f140-4fd4-b6b5-4e8b24e706d0",
  "callId": "vrs_ai_call_456",
  "soNo": "13538277",
  "occurredAt": "2026-07-24T17:24:00Z",
  "state": "partial",
  "authNumber": "AHS-002",
  "authorizedAmount": 250.0,
  "requestedAmount": 380.6,
  "denyReason": "Labor amount exceeded the coverage limit",
  "durationSec": 214,
  "recordingUrl": "https://vrs-dev.stage.nextgen.shs.com/api/vrs/call/456/recording"
}
```

Required outcome fields: `authNumber`, `authorizedAmount`, and `requestedAmount`. `denyReason`, `durationSec`, and `recordingUrl` are optional.

7.4 Denied

```
{
  "version": 1,
  "eventId": "7dc0deee-a2cb-47d4-996d-40fa12ed19cb",
  "callId": "vrs_ai_call_456",
  "soNo": "13538277",
  "occurredAt": "2026-07-24T17:24:00Z",
  "state": "denied",
  "denyReason": "Repair cost exceeds the authorization limit",
  "durationSec": 142,
  "recordingUrl": "https://vrs-dev.stage.nextgen.shs.com/api/vrs/call/456/recording"
}
```

`denyReason` is required. `durationSec` and `recordingUrl` are optional.

7.5 Failed

```
{
  "version": 1,
  "eventId": "1d62fbce-d5fe-46c0-91c1-46b11bc57ac5",
  "callId": "vrs_ai_call_456",
  "soNo": "13538277",
  "occurredAt": "2026-07-24T17:24:00Z",
  "state": "failed",
  "reason": "busy",
  "retryable": true
}
```

`reason` must be `busy`, `no_answer`, `ivr_error`, `agent_error`, `timeout`, or `aborted`. Under V5, `failed` is terminal even when `retryable` is true.

7.6 Requires information

```
{
  "version": 1,
  "eventId": "3b754582-e521-4d58-8bbe-e867202af9e5",
  "callId": "vrs_ai_call_456",
  "soNo": "13538277",
  "occurredAt": "2026-07-24T17:24:00Z",
  "state": "requires_info",
  "question": "What is the age of the unit?",
  "field": "appliance_age",
  "urgency": "hard"
}
```

`requires_info` is non-terminal and may be followed by one terminal event. For Phase 1, VRS AI owns calling `tech_phone` directly to obtain the answer. Express records and forwards the event but does not expose an app response endpoint.

7.7 Express business effects

VRS AI event	Express action
approved	Complete through the existing approve path using authNumber; send normal technician SMS
partial	Complete through the approve path; record and message the partial amount/details
denied	Reject through the existing ticket path; send normal rejection SMS
failed	Keep ticket available for manual follow-up; surface reason/retryable flag
requires_info	Status-only update; keep ticket pending and wait for later terminal event

Express then forwards the event to the upstream workflow with the public Express bridge callId. VRS AI does not perform that forwarding hop.

8. Callback signing

Calculate the signature over the exact UTF-8 bytes transmitted as the request body:

```
digest = HMAC-SHA256(callback_secret, exact_raw_request_body_bytes)
X-VRS-Signature = "sha256=" + lowercase_hex(digest)
```

Rules:

- Do not parse and reserialize the JSON after calculating the signature.
- Do not include the timestamp, URL, or headers in the HMAC input.
- `X-VRS-Event-Id` must exactly equal body `eventId`.
- `X-VRS-Timestamp` is current Unix epoch time in milliseconds.
- Express rejects timestamps more than five minutes from its clock.
- Use the same `eventId` and exact body on a retry.

- Refresh `X-VRS-Timestamp` on each retry attempt.
- Never use `X-API-Key` as the HMAC key. Use the request's `callback_secret`.

8.1 Deterministic test vector

This vector is for implementation testing only.

```
Secret:
vrs-ai-test-secret-do-not-use-in-prod

Exact raw body:
{"version":1,"eventId":"evt_001","callId":"vrs_ai_call_456","soNo":"13538277","occurredAt":"2026-07-
24T17:24:00Z","state":"approved","authNumber":"AHS-001","authorizedAmount":365.6}

Expected X-VRS-Signature:
sha256=5631d513c93853fa58efc382eed0c19db14ab42b94b1fa44938bb44ee352d4b6
```

Production `eventId` values should be UUID v4 even though this deterministic test vector uses a short fixed identifier.

9. Callback acknowledgement and retry

9.1 Success

```
HTTP/1.1 200 OK
Content-Type: application/json

{
  "ok": true,
  "eventId": "8f3a5e10-58a4-4fe0-a3a1-95038a7e4ee6",
  "deduped": false
}
```

For a repeated event ID, Express returns the same `200` shape with `deduped: true`. VRS AI must treat either response as delivery success.

9.2 Errors

Status	Example Express error	VRS AI action
400	VALIDATION_ERROR, EVENT_ID_MISMATCH	Do not retry unchanged payload; correct the producer
401	INVALID_SIGNATURE, TIMESTAMP_OUT_OF_WINDOW	Do not retry blindly; correct secret/signing/clock
500	INTERNAL	Retry
502 / 503	Transient proxy/configuration failure	Retry
Network timeout	No response	Retry

9.3 Retry policy

- Maximum 5 delivery attempts.
- Retry network failures and HTTP `500`, `502`, or `503`.
- Backoff after failed attempts: 1 second, 2 seconds, 4 seconds, 8 seconds.
- Preserve the same `eventId` and exact body across all attempts.
- Stop after any `200`.
- Do not retry unchanged `400` or `401` responses.

Express uses event-level apply/forward checkpoints. A delivery retry cannot repeat the ticket decision or technician SMS.

10. Event ordering and idempotency

- `requires_info` may precede one terminal event.
- Terminal states are `approved`, `partial`, `denied`, and `failed`.

- Once Express accepts a terminal event, VRS AI must not send a different terminal outcome for the same call.
- A retry uses the same event ID.
- `callId` must match the VRS AI provider call ID recorded from the authorize response.
- `soNo` must match the original request exactly.
- Unknown `callId` or mismatched `soNo` produces a transient `500` so a short dispatch/callback race can recover by retry.

11. Staging acceptance checklist

11.1 VRS AI readiness

- **[ ]** Staging `POST /vrs/authorize` is reachable at the confirmed hostname.
- **[ ]** A staging API key is provided to the Express team through a secure channel.
- **[ ]** `202`, `409`, `429`, `400`, and `401` responses match this document.
- **[ ]** `call_id` is stable and reused in every callback.
- **[ ]** Callback signing passes the deterministic test vector.
- **[ ]** Callback producer sends only the five Phase 1 event shapes.
- **[ ]** Callback retries use stable event IDs and the required backoff.
- **[ ]** AHS/Twilio call admission and limits are enforced by VRS AI.
- **[ ]** `cta_no` lookup and recording behavior are operational.

11.2 Express readiness

- **[ ] Staging credentials are newly provisioned, exchanged through an approved secure channel, and verified without appearing in source control or logs.**
- **[ ]** Additive bridge schema is applied to the development database only.
- **[ ]** `VRS_AI_BASE_URL`, `VRS_AI_API_KEY`, `VRS_AI_WEBHOOK_SECRET`, and `VRS_AI_CALLBACK_URL` are configured in staging.
- **[ ]** The Express callback URL is public, HTTPS, and ends exactly with `/api/integrations/vrs-ai/v1/webhook`.
- **[ ]** AHS test technician has an active account and a four-digit district.

11.3 Joint end-to-end evidence

Run and capture sanitized evidence for:

1. Approved call.
2. Partial approval.
3. Denied call.
4. `requires_info` followed by approval.
5. Retryable failed call.
6. Duplicate authorize returning `409` and `existing_call_id`.
7. Duplicate callback returning `200` and `deduped: true`.
8. Bad signature returning `401`.
9. Stale timestamp returning `401`.
10. Simulated Express `500` followed by a successful callback retry.

Capture HTTP status, Express bridge call ID, VRS AI call ID, event ID, state, and timestamps. Do not capture API keys, callback secrets, customer PII, or full phone numbers.

12. Decisions needed from VRS AI

Decision 1 - `repair_replace_rationale` prompt placement

Choose and document one:

1. Dedicated prompt section, separate from `comments[]`.
2. Merged into `tech_steps_summary`.

Recommendation: use a dedicated repair/replace rationale section so the technician's final recommendation is not diluted among chronological comments.

Decision 2 - retryable terminal failure ownership

V5 marks `failed` as terminal even when `retryable: true`. Confirm whether:

1. VRS AI automatically creates/redials a new call and emits a new `call_id`.
2. Express exposes an explicit human retry button that submits a new authorize request.

Express does not currently auto-redial.

Decision 3 - recording access

Confirm recording URL retention, authorization, and expiry. The V5 contract states recording access uses `X-API-Key`; Express currently stores the URL but does not fetch the recording.

13. Definition of integration-ready

The integration is ready for production review only when:

- both sides pass the deterministic signing vector;
- the full AHS staging flow passes from Express human approval through the final upstream callback;
- duplicate and retry scenarios prove no repeated ticket/SMS side effects;
- the three decisions above are recorded;
- secrets are rotated/configured without appearing in source control; and
- promotion occurs through the normal development-to-production pipeline.